

15 Working with Spring Data JDBC

This chapter covers

- Starting a Spring Data JDBC project
- Working with queries and query methods in Spring Data JDBC
- Building relationships using Spring Data JDBC
- Modeling embedded entities with Spring Data JDBC

We introduced Spring Data in chapter 2: it's an umbrella project containing many projects whose purpose is to simplify access to both relational and NoSQL databases by adhering to the Spring framework principles. In chapter 4, we looked in detail at the principles and capabilities of the Spring Data JPA project. The purpose of Spring Data JDBC is to efficiently deal with JDBC-based repositories. It is a younger project in the family, and it does not offer all the JPA capabilities, such as caching or lazy loading, resulting in a simpler and more limited ORM. However, it is growing and introducing new features with each version.

Why would we need Spring Data JDBC, when we already have alternatives such as JPA, Hibernate, and Spring Data JPA? The truth is that object/relational mapping (ORM) makes projects complex, and you clearly saw this in the previous chapters. There are situations when we'll want to eliminate this complexity and take the benefits of working with Spring, the most popular Java framework today. What alternatives do we have?

If we look back to the old-style JDBC, we have to remember its shortcomings, such as opening and closing connections by yourself or manually handling exceptions—overall, we would have to deal with a lot of service code.

Spring Data JDBC allows us to create our own queries to be executed against the database, but it also has its own ORM and uses concepts already used by JPA, Hibernate, and Spring Data JPA: entities, repositories, and `@Query` annotations. Spring Data JDBC does not use JPQL and there is no portability. Queries must be written in plain SQL and be specific to the database vendor. Entity loading must be done through SQL queries, and it is either complete or absent. Caching and lazy loading are not available. Sessions and dirty tracking do not exist; we have to explicitly save entities.

Also, at the time of writing this chapter, Spring Data JDBC does not support schema generation. We can declare our entities as we do in Hibernate or Spring Data JPA, but the DDL commands need to be written and run.

Let's create a project that uses Spring Data JDBC and analyze its capabilities to support us as we introduce new features.

15.1 Creating a Spring Data JDBC project

In this chapter, we'll create an application that will manage and persist CaveatEmptor users with Spring Data JDBC as a persistence framework, much like we did using Spring Data JPA in chapter 4. We'll create a Spring Boot application to use Spring Data JDBC.

To get started, we'll use the Spring Initializr website

(<https://start.spring.io/>) to create a new Spring Boot project (figure 15.1) with the following characteristics:

- Group: com.manning.javapersistence
- Artifact: spring-data-jdbc
- Description: Spring Data JDBC project

We'll also add the following dependencies:

- Spring Data JDBC (this will add `spring-boot-starter-data-jdbc` in the Maven pom.xml file)
- MySQL Driver (this will add `mysql-connector-java` in the Maven pom.xml file)

NOTE To execute the examples from the source code, you'll first need to run the Ch15.sql script.

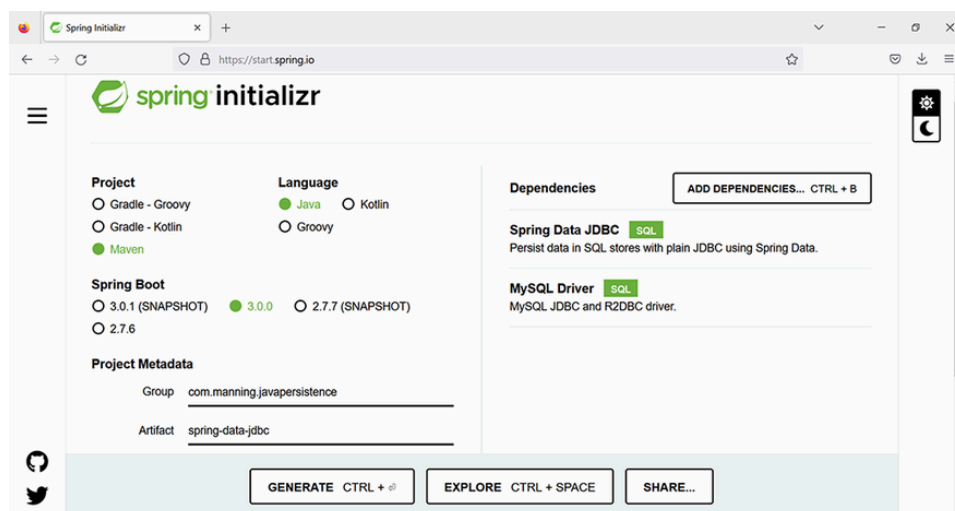


Figure 15.1 Creating a new Spring Boot project using Spring Data JDBC and MySQL

The skeleton of the project contains four files:

- SpringDataJdbcApplication, including a skeleton `main` method
- SpringDataJdbcApplicationTests, including a skeleton test method
- `application.properties`, which is empty at the beginning
- `pom.xml`, including the management information needed by Maven

The `pom.xml` file, shown in the following listing, includes the dependencies that we added to start the Spring Data JDBC project: we'll use the Spring Data JDBC framework to access a MySQL database, for which we need the driver.

Listing 15.1 The `pom.xml` Maven file

Path: `Ch15/spring-data-jdbc/pom.xml`

```
\1
<dependency>                                ①
  <groupId>org.springframework.boot</groupId>  ②
  <artifactId>spring-boot-starter-data-jdbc</artifactId>  ③
</dependency>                                ④

<dependency>                                ⑤
  <groupId>mysql</groupId>                    ⑥
  <artifactId>mysql-connector-java</artifactId>  ⑦
  <scope>runtime</scope>                      ⑧
</dependency>                                ⑨
<dependency>                                ⑩
  <groupId>org.springframework.boot</groupId>  ⑪
  <artifactId>spring-boot-starter-test</artifactId>  ⑫
  <scope>test</scope>                        ⑬
</dependency>                                ⑭
</dependencies>
```

① `spring-boot-starter-data-jdbc` is the starter dependency used by Spring Boot to connect to a relational database through Spring Data JDBC.

② `mysql-connector-java` is the JDBC driver for MySQL. It is a runtime dependency, meaning it is needed in the classpath only at runtime.

③ `spring-boot-starter-test` is the Spring Boot starter dependency for testing. It is needed only for the test compilation and execution phases.

The `application.properties` file can include various properties that will be used by the application. Spring Boot will automatically find and load the `application.properties` file from the classpath, and the `src/main/resources` folder is added by Maven to the classpath. As the initialization script is

run by default only for embedded databases, and we are using MySQL, we'll have to force the execution of the script by setting the initialization mode, `spring.sql.init.mode`, to be `always`. The configuration file is shown in the following listing.

Listing 15.2 The `application.properties` file

```
Path: Ch15/spring-data-jdbc/src/main/resources/application.properties

spring.datasource.url=jdbc:mysql://localhost:3306/CH15_SPRINGDATAJDBC
➡ ?serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
spring.sql.init.mode=always
```

- Ⓐ The URL of the database.
- Ⓑ The credentials to access the database. Replace them with the credentials on your machine, and use a password in real life.
- Ⓒ The dialect of the database, MySQL.
- Ⓓ The SQL initialization mode is `always`, so the SQL file will be always executed, recreating the database schema.

The SQL script that is automatically executed will look like the one in the following listing, dropping and recreating the `USERS` table. At startup, Spring Boot will always execute the `schema.sql` and `data.sql` files on the classpath.

Listing 15.3 The `schema.sql` file

```
Path: Ch15/spring-data-jdbc/src/main/resources/schema.sql

DROP TABLE IF EXISTS USERS;

CREATE TABLE USERS (
    ID INTEGER AUTO_INCREMENT PRIMARY KEY,
    USERNAME VARCHAR(30),
    REGISTRATION_DATE DATE
);
```

We'll now define the entity class corresponding to the `USERS` table as demonstrated in listing 15.4. We'll use some Spring-specific annotations to configure how the class is mapped to the table in the database:

- `org.springframework.data.relational.core.mapping.Table` —This is different from the previously used `javax.persistence.Table`, which is JPA-specific.
- `org.springframework.data.annotation.Id` —This is different from the previously used `javax.persistence.Id`, which is JPA-specific. We defined the corresponding column in the database as `ID INTEGER AUTO_INCREMENT PRIMARY KEY`, so the database will take care of generating the auto-incremented values.
- `org.springframework.data.relational.core.mapping.Column` —This is different from the previously used `javax.persistence.Column`, which is JPA-specific. For the column names, Spring Data JDBC will convert the camel case used for the definition of class fields into the snake case used for the definition of table columns.

Listing 15.4 The `User` class

Path: `Ch15/spring-data-jdbc/src/main/java/com/manning/javapersistence/c`
 ➡ `/model/User.java`

```

@Table("USERS")                                ❶
public class User {                             ❷

    @Id                                           ❸
    @Column("ID")                               ❹
    private Long id;

    @Column("USERNAME")                         ❺
    private String username;

    @Column("REGISTRATION_DATE")                ❻
    private LocalDate registrationDate;

    //constructors, getters and setters
}

```

❶ Annotate the `User` class with the `@Table` annotation, explicitly indicating that the corresponding table is `USERS`.

❷ Annotate the `id` field with the `@Id` annotation.

❸ Annotate the `id` field with the `@Column("ID")` annotation, specifying the corresponding column in the database. This is the default value.

❹ Annotate the `username` field with the `@Column("USERNAME")` annotation, specifying the corresponding column in the database. This is the default value.

❺ Annotate the `registrationDate` field with the `@Column("REGISTRATION_DATE")` annotation, specifying the corresponding column in

the database. This is the default value.

We'll also create the `UserRepository` interface that extends `CrudRepository` and thus provides access to the database.

Listing 15.5 The `UserRepository` interface

```
Path: Ch15/spring-data-jdbc/src/main/java/com/manning/javapersistence/c
➡ /repositories/UserRepository.java
```

```
@Repository
public interface UserRepository extends CrudRepository<User, Long> {
    List<User> findAll();
}
```

The `UserRepository` interface extends `CrudRepository<User, Long>`. This means that it is a repository of `User` entities, having a `Long` identifier. Remember, the `User` class has an `id` field annotated as `@Id` of type `Long`. We can directly call methods such as `save`, `findAll`, or `findById`, inherited from `CrudRepository`, and we can use them without any other additional information to execute the usual operations against a database. Spring Data JDBC will create a proxy class implementing the `UserRepository` interface and implement its methods.

NOTE It is worth recalling what we mentioned in section 4.3:

`CrudRepository` is a generic technology-agnostic persistence interface that we can use not only for JPA/relational databases, as you've seen so far.

We are only overriding the `findAll` method so that it returns `List<User>` instead of `Iterable<User>`. This will simplify our future tests. As a base class for all future tests, we'll write the `SpringDataJdbcApplicationTests` abstract class.

The `@SpringBootTest` annotation, added by Spring Boot to the initially created class, will tell Spring Boot to search the main configuration class (the `@SpringBootApplication` annotated class, for instance) and create the `ApplicationContext` to be used in the tests. As you'll recall, the `@SpringBootApplication` annotation added by Spring Boot to the class containing the `main` method will enable the Spring Boot autoconfiguration mechanism, enable the scan on the package where the application is located, and allow us to register extra beans in the context.

Using the `@TestInstance(TestInstance.Lifecycle.PER_CLASS)` annotation, we'll ask JUnit 5 to create a single instance of the test class and reuse it for all test methods. This will allow us to make the `@BeforeAll` and `@AfterAll` annotated methods non-static and to directly use inside

them the autowired `UserRepository` instance field. The `@BeforeAll` annotated non-static method is executed once before all tests from any class extending `SpringDataJdbcApplicationTests`, and it saves the list of users created inside the `generateUsers` method to the database. The `@AfterAll` annotated non-static method is executed once after all tests from any class extending `SpringDataJdbcApplicationTests`, and it deletes all users from the database.

Listing 15.6 The `SpringDataJdbcApplicationTests` abstract class

```
Path: Ch15/spring-data-jdbc/src/test/java/com/manning/javapersistence/c
➡ /SpringDataJdbcApplicationTests.java
```

```
@SpringBootTest
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
abstract class SpringDataJdbcApplicationTests {
    @Autowired                                ①
    UserRepository userRepository;            ①

    @BeforeAll
    void beforeAll() {
        userRepository.saveAll(generateUsers());
    }

    private static List<User> generateUsers() {
        List<User> users = new ArrayList<>();

        User john = new User("john", LocalDate.of(2020, Month.APRIL, 13

        //create and set a total of 10 users

        users.add(john);
        //add a total of 10 users to the list

        return users;
    }

    @AfterAll
    void afterAll() {
        userRepository.deleteAll();
    }
}
```

① Autowire a `UserRepository` instance. This is possible due to the `@SpringBootApplication` annotation, which enables the scan on the package where the application is located and registers the beans in the context.

The next tests will extend this class and use the already populated database. To test the methods that now belong to `UserRepository`, we'll create the `FindUsersUsingQueriesTest` class and follow the same recipe for writing tests: we'll call the repository method and verify its results.

Listing 15.7 The `FindUsersUsingQueriesTest` class

```
Path: Ch15/spring-data-jdbc/src/test/java/com/manning/javapersistence/c
➡ /FindUsersUsingQueriesTest.java

public class FindUsersUsingQueriesTest extends
➡ SpringDataJdbcApplicationTests{

    @Test
    void testFindAll() {
        List<User> users = userRepository.findAll();
        assertEquals(10, users.size());
    }
}
```

15.2 Working with queries in Spring Data JDBC

We'll now look at working with queries in Spring Data JDBC. We'll start by defining queries with the query builder mechanism, and move on to limiting query results, sorting and paging, streaming results, using modifying queries, and creating customized queries.

15.2.1 Defining query methods with Spring Data JDBC

We'll extend the `User` class by adding the fields `email`, `level`, and `active`. A user may have different levels, which will allow them to execute particular actions, such as bidding above some amount. A user may be active or retired (meaning they are no longer active in the CaveatEmptor auction system).

Our goal is to write a program that can address use cases that involve finding users with some particular level, users who are active or not, users with a given username or email, or users with a registration date in a given interval.

Listing 15.8 The modified `User` class

```
Path: Ch15/spring-data-jdbc2/src/main/java/com/manning/javapersistence/
➡ /model/User.java

@Table(name = "USERS")
public class User {

    @Id
```



```

        private Long id;

        private String username;

        private LocalDate registrationDate;

        private String email;

        private int level;

        private boolean active;

        //constructors, getters and setters
    }

```

As we are now responsible for the DDL commands to be executed, we can modify the content of the schema.sql file on the classpath.

Listing 15.9 The modified schema.sql file

Path: Ch15/spring-data-jdbc2/src/main/resources/schema.sql

```

DROP TABLE IF EXISTS USERS;

CREATE TABLE USERS (
    ID INTEGER AUTO_INCREMENT PRIMARY KEY,
    ACTIVE BOOLEAN,
    USERNAME VARCHAR(30),
    EMAIL VARCHAR(30),
    LEVEL INTEGER,
    REGISTRATION_DATE DATE
);

```

We'll now add new methods to the UserRepository interface that query the database, and we'll use them in newly created tests.

Listing 15.10 The UserRepository interface with new methods

Path: Ch15/spring-data-jdbc2/src/main/java/com/manning/javapersistence/
 ➡ /repositories/UserRepository.java

```

public interface UserRepository extends CrudRepository<User, Long> {
    List<User> findAll();
    Optional<User> findByUsername(String username);
    List<User> findAllOrderByUsernameAsc();
    List<User> findByRegistrationDateBetween(LocalDate start,
    ➡ LocalDate end);
    List<User> findByUsernameAndEmail(String username, String email);
    List<User> findByUsernameOrEmail(String username, String email);
    List<User> findByUsernameIgnoreCase(String username);
}

```

```

        List<User> findByLevelOrderByUsernameDesc(int level);
        List<User> findByLevelGreaterThanOrEqualTo(int level);
        List<User> findByUsernameContaining(String text);
        List<User> findByUsernameLike(String text);
        List<User> findByUsernameStartingWith(String start);
        List<User> findByUsernameEndingWith(String end);
        List<User> findByActive(boolean active);
        List<User> findByRegistrationDateIn(Collection<LocalDate> dates);
        List<User> findByRegistrationDateNotIn(Collection<LocalDate> dates);
        // . . .
    }

```

The purpose of the query methods is to retrieve information from the database. Starting with version 2.0, Spring Data JDBC provides a query builder mechanism similar to the one in Spring Data JPA—it creates the behavior of the repository methods based on their names. Remember that the query mechanism removes prefixes and suffixes such as `find...By`, `get...By`, `query...By`, `read...By`, and `count...By` from the name of the method and then parses the remainder of it.

Just like Spring Data JPA, Spring Data JDBC will look at the return type of the method. If we want to find a `User` and return it in an `Optional` container, the method return type will be `Optional<User>`.

The names of the methods need to follow the rules to determine the resulting query. Defined query methods may currently only use properties that can be included in a `WHERE` clause, but without joins. If the method naming is wrong (for example, the entity property does not match in the query method), we'll get an error when the application context is loaded. Table 15.1 summarizes the use of the essential keywords in building Spring Data JDBC query methods and the resulting conditions. For a more comprehensive list, see appendix C.

Table 15.1 Keyword usage in Spring Data JDBC and resulting conditions

Keyword	Example	Condition
Is, Equals	<pre>findByUsername(String name) findByUsernameIs(String name) findByUsernameEquals(String name)</pre>	<pre>username name name</pre>
And	<pre>findByUsernameAndRegistrationDate(String name, LocalDate date)</pre>	<pre>username name and reg: tra- tion date date</pre>
Or	<pre>findByUsernameOrRegistrationDate(String name, LocalDate date)</pre>	<pre>username name reg: tra- tion ev= name</pre>
LessThan	<pre>findByRegistrationDateLessThan(LocalDate date)</pre>	<pre>reg: tra- tion < d</pre>
LessThanEqual	<pre>findByRegistrationDateLessThanEqual(LocalDate date)</pre>	<pre>reg: tra- tion <= d</pre>
GreaterThan	<pre>findByRegistrationDateGreaterThan(LocalDate date)</pre>	<pre>reg: tra- tion > d</pre>
GreaterThanEqual	<pre>findByRegistrationDateGreaterThanEqual(LocalDate date)</pre>	<pre>reg: tra- tion >= d</pre>

Between	findByRegistrationDateBetween(LocalDate from, LocalDate to)	registration between from and
OrderBy	findByRegistrationDateOrderByUsernameDesc(LocalDate date)	registration = date ordered by username desc
Like	findByUsernameLike(String name)	username like name
NotLike	findByUsernameNotLike(String name)	username not like name
Before	findByRegistrationDateBefore(LocalDate date)	registration < date
After	findByRegistrationDateAfter(LocalDate date)	registration > date
Null, IsNull	findByRegistrationDate(Is)Null()	registration is
NotNull, IsNotNull	findByRegistrationDate(Is)NotNull()	registration

```
is
null

findByUsernameNot(String name)

use
name
name
```

We'll extend the `SpringDataJdbcApplicationTests` abstract class, the base class for our tests, by configuring the newly introduced fields `email`, `level`, and `active` for each user.

Listing 15.11 The updated `SpringDataJdbcApplicationTests` abstract class

```
Path: Ch15/spring-data-jdbc2/src/test/java/com/manning/javapersistence/
➡ /SpringDataJdbcApplicationTests.java

@SpringBootTest
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
abstract class SpringDataJdbcApplicationTests {
    // . . .
    private static List<User> generateUsers() {
        List<User> users = new ArrayList<>();

        User john = new User("john", LocalDate.of(2020, Month.APRIL, 13));
        john.setEmail("john@somedomain.com");
        john.setLevel(1);
        john.setActive(true);

        //create and set a total of 10 users

        users.add(john);
        //add a total of 10 users to the list

        return users;
    }

    // . . .
}
```

The next tests extend this class and use the already populated database. The use case we want to solve is getting a user or a list of users fulfilling a particular condition (such as a registration date within a given interval) or ordered by username. To test the methods that now belong to `UserRepository`, we'll create the `FindUsersUsingQueriesTest` class and follow the same recipe for writing tests: call the repository method and verify its results.

Listing 15.12 The `FindUsersUsingQueriesTest` class

Path: Ch15/spring-data-jdbc2/src/test/java/com/manning/javapersistence/
➡ /FindUsersUsingQueriesTest.java

```
public class FindUsersUsingQueriesTest extends
➡ SpringDataJdbcApplicationTests{

    @Test
    void testFindAll() {
        List<User> users = userRepository.findAll();
        assertEquals(10, users.size());
    }

    @Test
    void testFindUser() {
        User beth = userRepository.findByUsername("beth").get();
        assertEquals("beth", beth.getUsername());
    }

    @Test
    void testFindAllOrderByUsernameAsc() {
        List<User> users = userRepository.findAllOrderByUsernameAsc()
        assertAll(() -> assertEquals(10, users.size()),
            () -> assertEquals("beth", users.get(0).getUsername()),
            () -> assertEquals("stephanie",
                users.get(users.size() - 1).getUsername()));
    }

    @Test
    void testFindByRegistrationDateBetween() {
        List<User> users = userRepository.findByRegistrationDateBetween
            LocalDate.of(2020, Month.JULY, 1),
            LocalDate.of(2020, Month.DECEMBER, 31));
        assertEquals(4, users.size());
    }

    //more tests
}
```

15.2.2 Limiting query results, sorting, and paging

As in Spring Data JPA, the `first` and `top` keywords (used equivalently) can limit the results of query methods. The `top` and `first` keywords may be followed by an optional numeric value to indicate the maximum result size to be returned. If this numeric value is missing, the result size will be 1.

`Pageable` is an interface for pagination information. In practice, we use the `PageRequest` class that implements it. This class can specify the page number, the page size, and the sorting criterion.

The use cases we want to solve here are getting a limited number of users (such as the first user by username or by registration date) or the first users with a given level, sorted by registration date, or a large bunch of users in pages, so we can easily manipulate them.

We'll add the following methods to the `UserRepository` interface.

Listing 15.13 Limiting query results, sorting, and paging in `UserRepository`

```
Path: Ch15/spring-data-jdbc2/src/main/java/com/manning/javapersistence/
➡ /repositories/UserRepository.java
```

```
Optional<User> findFirstByOrderByUsernameAsc();
Optional<User> findTopByOrderByRegistrationDateDesc();
Page<User> findAll(Pageable pageable);
List<User> findFirst2ByLevel(int level, Sort sort);
List<User> findByLevel(int level, Sort sort);
List<User> findByActive(boolean active, Pageable pageable);
```

These methods follow the pattern required by the query builder mechanism (summarized in table 15.1), but this time their purpose is to limit the results of the query, so we can do sorting and paging. For example, the `Optional<User> findFirstByOrderByUsernameAsc()` method will get the first user by its username (the result is an `Optional`, so eventually it may not exist). The `Page<User> findAll(Pageable pageable)` method will get all users but in pages. We'll write the following tests to verify how these newly added methods work.

Listing 15.14 Testing limiting query results, sorting, and paging

```
Path: Ch15/spring-data-jdbc2/src/test/java/com/manning/javapersistence/
➡ /FindUsersSortingAndPagingTest.java
```

```
public class FindUsersSortingAndPagingTest extends
    SpringDataJdbcApplicationTests {

    @Test
    void testOrder() {

        User user1 = userRepository.findFirstByOrderByUsernameAsc().get()
        User user2 =
            userRepository.findTopByOrderByRegistrationDateDesc().get()
        Page<User> userPage = userRepository.findAll(PageRequest.of(1,
        List<User> users = userRepository.findFirst2ByLevel(2,
                                                    Sort.by("registrationDate")));

        assertAll(
            () -> assertEquals("beth", user1.getUsername()),
            () -> assertEquals("julius", user2.getUsername()),
```

```

        () -> assertEquals(2, users.size()),
        () -> assertEquals(3, userPage.getSize()),
        () -> assertEquals("beth", users.get(0).getUsername()),
        () -> assertEquals("marion", users.get(1).getUsername())
    );
}

@Test
void testFindByLevel() {
    Sort.TypedSort<User> user = Sort.sort(User.class);

    List<User> users = userRepository.findByLevel(3,
        user.by(User::getRegistrationDate).descending());
    assertEquals(
        () -> assertEquals(2, users.size()),
        () -> assertEquals("james", users.get(0).getUsername())
    );
}

@Test
void testFindByActive() {
    List<User> users = userRepository.findByActive(true,
        PageRequest.of(1, 4, Sort.by("registrationDate")));
    assertEquals(
        () -> assertEquals(4, users.size()),
        () -> assertEquals("burk", users.get(0).getUsername())
    );
}
}

```

- Ⓐ The first test finds the first user by ascending order of username and the first user by descending order of registration date.
- Ⓑ Find all users, split them into pages, and return page number 1 of size 3 (the page numbering starts with 0).
- Ⓒ Find the first 2 users with level 2, ordered by registration date.
- Ⓓ The second test defines a sorting criterion on the `User` class. `Sort.TypedSort` extends `Sort` and can use method handles to define properties to sort by.
- Ⓔ Find users of level 3 and sort by registration date, descending.
- Ⓕ The third test finds the active users sorted by registration date, splits them into pages, and returns page number 1 of size 4 (page numbering starts with 0).

15.2.3 Streaming results

Query methods returning more than one result can use standard Java interfaces, such as `Iterable`, `List`, and `Set`. Like Spring Data JPA, Spring Data JDBC supports `Streamable`, which can be used as an alternative to `Iterable` or any collection type. It allows us to concatenate `Streamable`s and to directly filter and map over the elements.

The use case we'll solve here is getting the results as a stream, without waiting for the whole collection of users or a page of users to be obtained. This way, we can quickly start processing the first results as they flow in. Unlike collections, a stream can be consumed only once and is immutable.

We'll add the following methods to the `UserRepository` interface.

Listing 15.15 Adding methods that return `Streamable` in `UserRepository`

```
Path: Ch15/spring-data-jdbc2/src/main/java/com/manning/javapersistence/  
➡ /repositories/UserRepository.java
```

```
Streamable<User> findByEmailContaining(String text);  
Streamable<User> findByLevel(int level);
```

We'll write the following test to verify how these newly added methods interact with the database and provide results as streams. A stream is given as a resource of the `try` block, so it is automatically closed. An alternative is to explicitly call the `close()` method. Otherwise the stream would keep the underlying connection to the database.

Listing 15.16 Testing methods that return `Streamable`

```
Path: Ch15/spring-data-jdbc2/src/test/java/com/manning/javapersistence/  
➡ /QueryResultsTest.java
```

```
@Test  
void testStreamable() {  
    try(Stream<User> result =  
        userRepository.findByEmailContaining("someother")  
        .and(userRepository.findByLevel(2))  
        .stream().distinct()) {  
        assertEquals(6, result.count());  
    }  
}
```

Ⓐ The test will call the `findByEmailContaining` method to search for emails containing the `someother` word.

- Ⓑ The test will concatenate the resulting `Streamable` with the `Streamable` providing users of level 2.
- Ⓒ It will transform this into a stream and will keep the distinct users.
- Ⓓ Check that the resulting stream contains 6 users.

15.2.4 The `@Query` annotation

We can use the `@Query` annotation to create methods for which a custom query can be specified. With the `@Query` annotation, the method name does not need to follow any naming convention. The custom query can be parameterized, but unlike for Spring Data JPA, the parameters can be identified only by name, and they must be bound in the query with the `@Param` annotation. Unlike with Spring Data JPA, we do not use JPQL but SQL. Consequently, there is no portability—if you change the database vendor, you will have to rewrite the queries.

We'll add two new methods to the `UserRepository` interface. These methods will be annotated with the `@Query` annotation, and their generated behavior will depend on the definition of these queries.

Listing 15.17 Methods annotated with `@Query` in the `UserRepository` interface

```
Path: Ch15/spring-data-jdbc2/src/main/java/com/manning/javapersistence/
➡ /repositories/UserRepository.java
```

```
@Query("SELECT COUNT(*) FROM USERS WHERE ACTIVE = :ACTIVE")
int findNumberOfUsersByActivity(@Param("ACTIVE") boolean active);

@Query("SELECT * FROM USERS WHERE LEVEL = :LEVEL AND ACTIVE = :ACTIVE")
List<User> findByLevelAndActive(@Param("LEVEL") int level, @Param("ACTIVE")
                               boolean active);
```

- Ⓐ The `findNumberOfUsersByActivity` method will return the number of active users.
- Ⓑ The `findByLevelAndActive` method will return users with the `level` and `active` status given as named parameters. The `@Param` annotation will match the `:LEVEL` parameter of the query with the `level` argument of the method and the `:ACTIVE` parameter of the query with the `active` argument of the method.

Writing tests for these query methods is pretty straightforward and similar to the previous examples. They can be found in the source code for the book.

15.2.5 Modifying queries

We can define modifying methods with the `@Modifying` annotation. For example, `INSERT`, `UPDATE`, and `DELETE` queries and DDL statements modify the content of the database. The `@Query` annotation can take as an argument the modifying query, and it may need binding parameters. At the time of writing, Spring Data JDBC does not support query derivation for delete methods (unlike Spring Data JPA).

We'll add the new methods, annotated with the `@Query` annotation, to the `UserRepository` interface, but this time the queries will update or delete records from the `USERS` table.

Listing 15.18 Adding modifying methods to the `UserRepository` interface

```
Path: Ch15/spring-data-jdbc2/src/main/java/com/manning/javapersistence/  
➡ /repositories/UserRepository.java
```

```
@Modifying  
@Query("UPDATE USERS SET LEVEL = :NEW_LEVEL WHERE LEVEL = :OLD_LEVEL")  
int updateLevel(@Param("OLD_LEVEL") int oldLevel,  
➡ @Param("NEW_LEVEL") int newLevel); ①
```

```
@Modifying  
@Query("DELETE FROM USERS WHERE LEVEL = :LEVEL")  
int deleteByLevel(@Param("LEVEL") int level); ②
```

① The `updateLevel` method will change the `level` of the users having the `oldLevel` and set it to the `newLevel`. The method is also annotated with `@Modifying`.

② The `deleteByLevel` method will remove all users with the `level` given as a parameter, as the argument of the `@Query` annotation indicates. The method is also annotated with `@Modifying`.

Writing the tests for these query methods is pretty straightforward and similar to the previous examples. They can be found in the source code for the book.

15.3 Modeling relationships with Spring Data JDBC

Managing the associations between classes and the relationships between tables is at the heart of ORM problems. We examined the possible solutions to these problems using JPA and Spring Data JPA in chapter 8, and we'll now look at the approaches provided by Spring Data JDBC.

15.3.1 Modeling a one-to-one relationship with Spring Data JDBC

Spring Data JPA can model relationships between entities with the JPA annotations `@OneToOne`, `@OneToMany`, `@ManyToMany`. Spring Data JDBC uses a different mechanism than JPA. We'll start with modeling one-to-one relationships between entities in Spring Data JDBC using the `User` and `Address` entities. Each `User` will have only one `Address`, and each `Address` will belong to one `User`.

As previously stated, Spring Boot will always execute the `schema.sql` file on the classpath at startup. As shown in the following listing, it drops and recreates the `ADDRESSES` and `USERS` tables.

Listing 15.19 The `schema.sql` file for one-to-one relationships

```
Path: Ch15/spring-data-jdbc3/src/main/resources/schema.sql
```

```
DROP TABLE IF EXISTS ADDRESSES;
DROP TABLE IF EXISTS USERS;

CREATE TABLE USERS (
    ID INTEGER AUTO_INCREMENT PRIMARY KEY,
    ACTIVE BOOLEAN,
    USERNAME VARCHAR(30),
    EMAIL VARCHAR(30),
    LEVEL INTEGER,
    REGISTRATION_DATE DATE
);

CREATE TABLE ADDRESSES (
    USER_ID INTEGER AUTO_INCREMENT PRIMARY KEY,
    STREET VARCHAR(30) NOT NULL,
    CITY VARCHAR(20) NOT NULL
);
```

The `@MappedCollection` annotation (introduced in Spring Data JDBC 1.1) can be used on a reference type for a one-to-one relationship. The `ID` field of the `USERS` table will be a foreign key in the `ADDRESSES` table, the corresponding field from the `ADDRESSES` table being `USER_ID`. Having one single `Address` reference inside `User` will make the relationship one-to-one. In the `User` class, the reference to the `Address` field will look like this:

```
Path: Ch15/spring-data-jdbc3/src/main/java/com/manning/javapersistence/
➡ /model/User.java
```

```
@Table("USERS")
public class User {
```

```

@Id
private Long id;

// . . .

@MappedCollection(idColumn = "USER_ID")
private Address address;

```

The `Address` class will also be annotated with `@Table`, because it will correspond to a different table from the database:

```

Path: Ch15/spring-data-jdbc3/src/main/java/com/manning/javapersistence/
➡ /model/Address.java

```

```

@Table("ADDRESSES")
public class Address {
// . . .

```

We'll create two repositories. The first is for the `User` entity:

```

Path: Ch15/spring-data-jdbc3/src/main/java/com/manning/javapersistence/
➡ /repositories/UserOneToOneRepository.java

```

```

public interface UserOneToOneRepository extends
➡ CrudRepository<User, Long> {

}

```

The second is for the `Address` entity:

```

Path: Ch15/spring-data-jdbc3/src/main/java/com/manning/javapersistence/
➡ /repositories/AddressOneToOneRepository.java

```

```

public interface AddressOneToOneRepository extends
        CrudRepository<Address, Long> {

}

```

We'll use these repositories to populate the database and execute the tests:

```

Path: Ch15/spring-data-jdbc3/src/test/java/com/manning/javapersistence/
➡ /UserAddressOneToOneTest.java

```

```

@SpringBootTest
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
public class UserAddressOneToOneTest {

    @Autowired

```

```

        private UserOneToOneRepository userOneToOneRepository;

        @Autowired
        private AddressOneToOneRepository addressOneToOneRepository;

        // . . .

        @Test
        void oneToOneTest() {

            assertAll(
                () -> assertEquals(10, userOneToOneRepository.count()),
                () -> assertEquals(10, addressOneToOneRepository.count(
            );

        }
        // . . .
    }

```

15.3.2 Modeling embedded entities with Spring Data JDBC

We now move on to the modeling of embedded entities in Spring Data JDBC. We want to embed the `User` entity and the `Address` class in `User`.

The SQL script that's automatically executed is shown in the following listing. There will be only one table, `USERS`, which will embed the information about addresses.

Listing 15.20 The `schema.sql` file for embedded entities

Path: `Ch15/spring-data-jdbc4/src/main/resources/schema.sql`

```

DROP TABLE IF EXISTS USERS;

CREATE TABLE USERS (
    ID INTEGER AUTO_INCREMENT PRIMARY KEY,
    ACTIVE BOOLEAN,
    USERNAME VARCHAR(30),
    EMAIL VARCHAR(30),
    LEVEL INTEGER,
    REGISTRATION_DATE DATE,
    STREET VARCHAR(30) NOT NULL,
    CITY VARCHAR(20) NOT NULL
);

```

The address will be embedded in the `USERS` table. If the embedded `STREET` and `CITY` columns are empty, the `address` field is `null`. In the `User` class, the reference to the `Address` field will look like this:

```
Path: Ch15/spring-data-jdbc4/src/main/java/com/manning/javapersistence/
➡ /model/User.java
```

```
@Table("USERS")
public class User {

    @Id
    private Long id;

    // . . .

    @Embedded(onEmpty = Embedded.OnEmpty.USE_NULL)
    private Address address;
```

The `Address` class will not be annotated with `@Table`, as it will no longer correspond to a different table from the database—all its information will be embedded in the `USERS` table.

```
Path: Ch15/spring-data-jdbc4/src/main/java/com/manning/javapersistence/
➡ /model/Address.java
```

```
public class Address {
    // . . .
```

We'll create a single repository for the `User` entity.

```
Path: Ch15/spring-data-jdbc4/src/main/java/com/manning/javapersistence/
➡ /repositories/UserAddressEmbeddedRepository.java
```

```
public interface UserAddressEmbeddedRepository extends
    CrudRepository<User, Long> {

}
```

We'll then use this repository to populate the database and execute the tests.

```
Path: Ch15/spring-data-jdbc4/src/test/java/com/manning/javapersistence/
➡ /UserAddressEmbeddedTest.java
```

```
@SpringBootTest
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
public class UserAddressEmbeddedTest {

    @Autowired
    private UserAddressEmbeddedRepository userAddressEmbeddedRepository

    // . . .
```

```

@Test
void embeddedTest() {

    assertEquals(10, userAddressEmbeddedRepository.count());

}
// . . .
}

```

15.3.3 Modeling a one-to-many relationship with Spring Data JDBC

We'll now move on to the modeling of a one-to-many relationship in Spring Data JDBC. We have the `User` entity and the `Address` entity. Each user can have many addresses.

The SQL script that's automatically executed is shown in the following listing. There will be two tables, `USERS` and `ADDRESSES`.

Listing 15.21 The `schema.sql` file for one-to-many relationships

Path: `Ch15/spring-data-jdbc5/src/main/resources/schema.sql`

```

DROP TABLE IF EXISTS ADDRESSES;
DROP TABLE IF EXISTS USERS;

CREATE TABLE USERS (
    ID INTEGER AUTO_INCREMENT PRIMARY KEY,
    ACTIVE BOOLEAN,
    USERNAME VARCHAR(30),
    EMAIL VARCHAR(30),
    LEVEL INTEGER,
    REGISTRATION_DATE DATE
);

CREATE TABLE ADDRESSES (
    ID INTEGER AUTO_INCREMENT PRIMARY KEY,
    USER_ID INTEGER,
    STREET VARCHAR(30) NOT NULL,
    CITY VARCHAR(20) NOT NULL,
    FOREIGN KEY (USER_ID)
        REFERENCES USERS(ID)
        ON DELETE CASCADE
);

```

The `ID` field of the `USERS` table will be a foreign key in the `ADDRESSES` table, the corresponding field from the `ADDRESSES` table being `USER_ID`. Having a set of `Address` references inside `User` will indicate that a `User` has many `Address`es. In the `User` class, the reference to the `Address`es will look like this:


```
Path: Ch15/spring-data-jdbc5/src/main/java/com/manning/javapersistence/  
➡ /model/User.java
```

```
@Table("USERS")  
public class User {  
  
    @Id  
    private Long id;  
  
    // . . .  
  
    @MappedCollection(idColumn = "USER_ID")  
    private Set<Address> addresses = new HashSet<>();  
}
```

The `Address` class will also be annotated with `@Table`, as it will correspond to a different table from the database:

```
Path: Ch15/spring-data-jdbc5/src/main/java/com/manning/javapersistence/  
➡ /model/Address.java
```

```
@Table("ADDRESSES")  
public class Address {  
    // . . .  
}
```

We'll create two repositories: one for the `User` entity and one for the `Address` entity. The second repository will contain an additional method. Even if the name of the `countByUserId` method follows the pattern discussed both for Spring Data JDBC and Spring Data JPA, the method needs to be annotated with `@Query`, as `userId` does not exist in the `Address` class:

```
Path: Ch15/spring-data-jdbc5/src/main/java/com/manning/javapersistence/  
➡ /repositories/AddressOneToManyRepository.java
```

```
public interface AddressOneToManyRepository  
    extends CrudRepository<Address, Long> {  
  
    @Query("SELECT COUNT(*) FROM ADDRESSES WHERE USER_ID = :USER_ID")  
    int countByUserId(@Param("USER_ID") Long userId);  
}
```

We'll use the repositories to populate the database and execute the tests:

```
Path: Ch15/spring-data-jdbc5/src/test/java/com/manning/javapersistence/  
➡ /UserAddressOneToManyTest.java
```

```
@SpringBootTest  
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
```

```

public class UserAddressOneToManyTest {

    @Autowired
    private UserOneToManyRepository userOneToManyRepository;

    @Autowired
    private AddressOneToManyRepository addressOneToManyRepository;

    // . . .

    @Test
    void oneToManyTest() {

        assertAll(
            () -> assertEquals(10, userOneToManyRepository.count()),
            () -> assertEquals(20, addressOneToManyRepository.count()),
            () -> assertEquals(2,

➡ addressOneToManyRepository.countByUserId(users.get(0).getId())
            );

        }
        // . . .
    }
}

```

15.3.4 Modeling a many-to-many relationship with Spring Data JDBC

We'll now move on to modeling a many-to-many relationship in Spring Data JDBC. We have the `User` and `Address` entities. Each `User` can have many `Address`s, and each `Address` may have many `User`s. We'll also need to manually introduce a class corresponding to the `USERS_ADDRESSES` intermediary table, which will model the many-to-many relationship.

The SQL script that's automatically executed will look like the following listing. There will be three tables: `USERS`, `ADDRESSES`, and `USERS_ADDRESSES`.

Listing 15.22 The `schema.sql` file for many-to-many relationships

Path: `Ch15/spring-data-jdbc6/src/main/resources/schema.sql`

```

DROP TABLE IF EXISTS USERS_ADDRESSES;
DROP TABLE IF EXISTS USERS;
DROP TABLE IF EXISTS ADDRESSES;

CREATE TABLE USERS (
    ID INTEGER AUTO_INCREMENT PRIMARY KEY,
    ACTIVE BOOLEAN,
    USERNAME VARCHAR(30),
    EMAIL VARCHAR(30),

```

```

        LEVEL INTEGER,
        REGISTRATION_DATE DATE
    );

CREATE TABLE ADDRESSES (
    ID INTEGER AUTO_INCREMENT PRIMARY KEY,
    STREET VARCHAR(30) NOT NULL,
    CITY VARCHAR(20) NOT NULL
);

CREATE TABLE USERS_ADDRESSES (
    USER_ID INTEGER,
    ADDRESS_ID INTEGER,
    FOREIGN KEY (USER_ID)
        REFERENCES USERS(ID)
        ON DELETE CASCADE,
    FOREIGN KEY (ADDRESS_ID)
        REFERENCES ADDRESSES(ID)
        ON DELETE CASCADE
);

```

To model many-to-many relationships, the `User` class will be connected to the intermediary class `UserAddress`. Having a set of `UserAddress` references inside `User` will indicate that a `User` has many `UserAddress`s. The `ID` field of the `USERS` table will be a foreign key in the `USERS_ADDRESSES` table, the corresponding field from the `USERS_ADDRESSES` table being `USER_ID`. In the `User` class, the reference to the `Addresses` will look like this:

```

Path: Ch15/spring-data-jdbc6/src/main/java/com/manning/javapersistence/
➡ /model/User.java

@Table("USERS")
public class User {

    @Id
    private Long id;

    // . . .

    @MappedCollection(idColumn = "USER_ID")
    private Set<UserAddress> addresses = new HashSet<>();
}

```

The `Address` class will also be annotated with `@Table`, as it will correspond to a different table from the database:

```

Path: Ch15/spring-data-jdbc6/src/main/java/com/manning/javapersistence/
➡ /model/Address.java

@Table("ADDRESSES")

```

```
public class Address {
    // . . .
```

Also, we'll create the `UserAddress` class and annotate it with `@Table`, as it will correspond to a different table from the database. It will hold only the ID of the `Address`, as the `User` class keeps a set of references of `UserAddress` type.

```
Path: Ch15/spring-data-jdbc6/src/main/java/com/manning/javapersistence/
➡ /model/UserAddress.java
```

```
@Table("USERS_ADDRESSES")
public class UserAddress {
    private Long addressId;

    public UserAddress(Long addressId) {
        this.addressId = addressId;
    }

    public Long getAddressId() {
        return addressId;
    }
}
```

We'll create three repositories: one for the `User` entity, one for the `Address` entity, and one for the `UserAddress` entity. The third repository will contain an additional method: even though the name of the `countByUserId` method follows the pattern discussed both for Spring Data JDBC and Spring Data JPA, the method needs to be annotated with `@Query`, because the `userId` does not exist in the `UserAddress` class.

```
Path: Ch15/spring-data-jdbc6/src/main/java/com/manning/javapersistence/
➡ /repositories/UserAddressManyToManyRepository.java
```

```
public interface UserAddressManyToManyRepository extends
    CrudRepository<UserAddress, Long> {

    @Query("SELECT COUNT(*) FROM USERS_ADDRESSES WHERE USER_ID = :USER_
        int countByUserId(@Param("USER_ID") Long userId);

}
```

We'll use the repositories to populate the database and execute the tests:

```
Path: Ch15/spring-data-jdbc6/src/test/java/com/manning/javapersistence/
➡ /UserAddressManyToManyTest.java
```

```
@SpringBootTest
```

```

@TestInstance(TestInstance.Lifecycle.PER_CLASS)
public class UserAddressManyToManyTest {

    @Autowired
    private UserAddressManyToManyRepository
    ➡ userAddressManyToManyRepository;

    @Autowired
    private AddressManyToManyRepository addressManyToManyRepository;

    @Autowired
    private UserManyToManyRepository userManyToManyRepository;

    // . . .

    @Test
    void manyToManyTest() {

        assertAll(
            () -> assertEquals(10, userManyToManyRepository.count()),
            () -> assertEquals(3, addressManyToManyRepository.count()),
            () -> assertEquals(20,
                ➡ userAddressManyToManyRepository.count()),
            () -> assertEquals(2,
                userAddressManyToManyRepository.countByUserId(
                                                            users.get(0).getId())
            );

        }

    // . . .
}

```

To conclude and finally compare and contrast the current capabilities of Spring Data JPA and Spring Data JDBC, take a look at table 15.2. We've summarized here the most important capabilities, such as portability, the complexity of learning and adopting it in projects, query derivation, native SQL usage, annotation usage, relationship modeling, caching and lazy loading, sessions, and dirty tracking.

Table 15.2 Spring Data JPA capabilities vs. Spring Data JDBC capabilities

Spring Data JPA	Spring Data JDBC
Database-independent and portable	Generally, database-specific
Introduces complexity through object/relational mapping (ORM)	Less complex, but still adheres to the Spring Framework principles
Automatic schema generation based on the entities	Schema generation through DDL commands on the side of the programmer
Query derivation since the first version	Query derivation since version 2.0
Queries annotated with JPQL code and native SQL	Only queries using native SQL
Can reuse classes with JPA annotations	Uses annotations from the <code>org.springframework.data</code> package
Models relationships between entities through annotations such as <code>@OneToMany</code> , <code>@Embedded</code> , etc.	Models relationships mostly on the side of the programmer, through the design of the classes
Caching and lazy loading	No caching, no lazy loading
Sessions and dirty tracking	No sessions, no dirty tracking

NOTE Spring Data JDBC is a young project, and it is in full development. It is expected to add considerable new features in the near future.

Summary

- You can create and configure a Spring Data JDBC project using Spring Boot, adding step-by-step methods that query the database and model different types of relationships.
- You can define and use a series of query methods to access repositories by following the Spring Data JDBC query builder mechanisms, as introduced in version 2.0 of the framework.
- Spring Data JDBC capabilities include limiting query results, sorting, paging, streaming results, and the `@Query` annotation.

- You can create and use modifying queries to update and delete entities.
- You can model one-to-one, one-to-many, and many-to-many relationships between entities as well as embedded entities with Spring Data JDBC.